

```
"j_posts_tags_tag_pk_fkey" FOREIGN KEY (tag_pk) REFERENCES
tags(pk)
```

2. First, let's add a primary key to the j_posts_add table:

```
forumbd=> alter table j_posts_tags add constraint j_posts_tags_pkey
primary key (tag_pk,post_pk);
ALTER TABLE

ALTER TABLE

forumbd=> \d j_posts_tags
          Table "forum.j_posts_tags"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 tag_pk | integer |           | not null |
 post_pk | integer |           | not null |
Indexes:
    "j_posts_tags_pkey" PRIMARY KEY, btree (tag_pk, post_pk)
Foreign-key constraints:
    "j_posts_tags_post_pk_fkey" FOREIGN KEY (post_pk) REFERENCES
posts(pk)
    "j_posts_tags_tag_pk_fkey" FOREIGN KEY (tag_pk) REFERENCES
tags(pk)
```

3. Next, let's insert some records in the j_posts_tags table:

```
forumbd=> insert into j_posts_tags (post_pk ,tag_pk) values
(3,2),(1,1),(2,1);
INSERT 0 3

forumbd=> select * from j_posts_tags ;
 tag_pk | post_pk
-----+-----
       2 |       3
       1 |       1
       1 |       2
(3 rows)
```

4. Now let's try to insert another record with the same primary key. If we perform a standard insert statement, as follows, we can see that PostgreSQL returns an error because we are trying to insert a record that already exists:

```
forumdb=>insert into j_posts_tags (post_pk ,tag_pk) values (2,1);
ERROR:  duplicate key value violates unique constraint "j_posts_
tags_pkey"
DETAIL:  Key (tag_pk, post_pk)=(1, 2) already exists.
```

5. Let's now try using the ON CONFLICT DO NOTHING option:

```
forumdb=> insert into j_posts_tags (post_pk ,tag_pk) values (2,1) ON
CONFLICT DO NOTHING;
INSERT 0 0

forumdb=> select * from j_posts_tags ;
tag_pk | post_pk
-----+-----
      2 |      3
      1 |      1
      1 |      2
(3 rows)
```

In this case, PostgreSQL doesn't return an error; instead, it simply does nothing.

6. Now let's try the DO UPDATE set option. This option realizes the UPSERT statement, as in the following example:

```
forumdb=> insert into j_posts_tags (post_pk ,tag_pk) values (2,1) ON
CONFLICT (tag_pk,post_pk) DO UPDATE set tag_pk=excluded.tag_pk+1;
INSERT 0 1

forumdb=> select * from j_posts_tags ;
tag_pk | post_pk
-----+-----
      2 |      3
      1 |      1
      2 |      2
(3 rows)
```

The fields inside the ON CONFLICT condition must have a unique or exclusion constraint. The previous statement simply replaces the following statement:

```
INSERT INTO j_posts_tags (post_pk ,tag_pk) values (2,1)
```

It gets replaced with this statement:

```
UPDATE set tag_pk=tag_pk+1 where tag_pk=1 and post_pk=2
```

Learning the RETURNING clause for INSERT

In PostgreSQL, we can add the RETURNING keyword to the insert statement. The RETURNING keyword in PostgreSQL provides an opportunity to return the values of any columns from an insert or update statement after the insert or update was run. For example, if we want to return all the fields of the record that we have just inserted, we have to perform a query as follows:

```
forumbd=> insert into j_posts_tags (tag_pk,post_pk) values(1,3) returning
*;
tag_pk | post_pk
-----+-----
      1 |      3
(1 row)

INSERT 0 1
```

The * means that we want to return all the fields of the record that we have just inserted; if we want to return only some fields, we have to specify what fields the query has to return:

```
forumbd=> insert into j_posts_tags (tag_pk,post_pk) values(1,2) returning
tag_pk;
tag_pk
-----
      1
(1 row)

INSERT 0 1
```

This feature will show itself to be particularly useful at the end of the chapter when we talk about CTEs.

Returning tuples out of queries

In previous chapters, we have looked at simple update queries, such as the following:

```
forumdb=> update posts set title = 'A view of Data types in C++' where pk
= 3;
UPDATE 1
```

Now we will look at something more complicated. What if we want to update some records in the posts table that are related in some way?

UPDATE related to multiple records

Let's start with the following scenario:

1. Consider the categories table:

```
forumdb=> SELECT * FROM categories;
pk |          title          |          description
----+-----+-----
 1 | Database                | Database related discussions
 2 | Unix                   | Unix and Linux discussions
 3 | Programming Languages   | All about programming languages
 4 | New Category           |
 5 | Database                | PostgreSQL
(5 rows)
```

2. Let's consider a new table of categories from which we want to update the existing categories table.

```
forumdb=> create temp table t_categories as select * from categories
limit 0;
SELECT 0

forumdb=> insert into t_categories (pk,title,description) values
(4,'Machine Learning','Machine Learning discussions'),(5,'Software
engineering','Software engineering discussions');
INSERT 0 2

forumdb=> select * from t_categories ;
pk |          title          |          description
----+-----+-----
```

```

4 | Machine Learning      | Machine Learning discussions
5 | Software engineering  | Software engineering discussions
(2 rows)

```

Let's suppose we want to pick up the values from the table `t_categories` and use them to update the values of the table `categories`; here is the resultset we want to reach:

```

pk |          title          |          description
----+-----+-----
1 | Database                | Database related discussions
2 | Unix                    | Unix and Linux discussions
3 | Programming Languages   | All about programming languages
4 | Machine Learning        | Machine Learning discussions
5 | Software engineering    | Software engineering discussions

```

The query we have to execute is:

```

forumdb=>update categories c set title=t.title,description=t.description
from t_categories t where c.pk=t.pk;
UPDATE 2

forumdb=> select * from categories;
pk |          title          |          description
----+-----+-----
1 | Database                | Database related discussions
2 | Unix                    | Unix and Linux discussions
3 | Programming Languages   | All about programming languages
4 | Machine Learning        | Machine Learning discussions
5 | Software engineering    | Software engineering discussions
(5 rows)

```

In this query, PostgreSQL is able to update the fields `title` and `description` of the `categories` table using the data from the table `t_categories` that have a match on the `pk` field; when we talk about the `merge` statement, we'll see another way to reach the same goal.

MERGE

Starting from PostgreSQL 15, we can achieve the same goal we achieved in the previous section by using the `MERGE` statement; it is preferable to use the `MERGE` statement as it is present in SQL 2003 ANSI.

Now, let's start from the previous values of table categories:

```

forumbd=> select * from categories;
pk |          title          |          description
----+-----+-----
  1 | Database                | Database related discussions
  2 | Unix                    | Unix and Linux discussions
  3 | Programming Languages   | All about programming languages
  4 | Machine Learning        | Machine Learning discussions
  5 | Software engineering    | Software engineering discussions
(5 rows)

```

Then let's create another dataset with some changes that we want to apply to the categories table:

```

forumbd=> create temp table new_data as select * from categories limit 0;
SELECT 0

forumbd=> insert into new_data (pk,title,description) values (1,'Database
Discussions','Database discussions'),(2,'Unix/Linux discussion','Unix and
Linux discussions');
INSERT 0 2

forumbd=> select * from new_data;
pk |          title          |          description
----+-----+-----
  1 | Database Discussions    | Database discussions
  2 | Unix/Linux discussion  | Unix and Linux discussions
(2 rows)

```

Now the goal we want to achieve is to merge the two datasets as shown below:

```

  1 | Database Discussions    | Database discussions
  2 | Unix/Linux discussion  | Unix and Linux discussions
  3 | Programming Languages   | All about programming languages
  4 | Machine Learning        | Machine Learning discussions
  5 | Software engineering    | Software engineering discussions

```

The query that we have to perform to reach this goal is:

```

forumbd=> merge into categories c
using new_data n on c.pk=n.pk
when matched then
    update set title=n.title,description=n.description
when not matched then
    insert (pk,title,description)
    OVERRIDING SYSTEM VALUE values (n.pk,n.title,n.description);
MERGE 2

forumbd=> select * from categories order by 1;
pk |          title          |          description
----+-----+-----
  1 | Database Discussions    | Database discussions
  2 | Unix/Linux discussion  | Unix and Linux discussions
  3 | Programming Languages   | All about programming languages
  4 | Machine Learning       | Machine Learning discussions
  5 | Software engineering    | Software engineering discussions
(5 rows)

```

The query above checks if there is a match between the value of the field PK of the new_data table and the value of the field of the categories table. If there is a match, the UPDATE will be executed; otherwise, the INSERT will be executed. The OVERRIDING SYSTEM VALUE clause is used because, in the INSERT statement, we have also specified the insertion of the values of the PK field taken from the new_data table, and since the PK field in the categories table is defined as GENERATED ALWAYS, without the OVERRIDING SYSTEM VALUE clause, PostgreSQL will generate an error.

Exploring UPDATE ... RETURNING

As with the INSERT statement, the update statement also has the possibility of adding the RETURNING keyword. The update statement works in the same way as the INSERT statement:

```

forumbd=> update categories set title='A.I' where pk=4 returning
pk,title,description;
pk | title |          description
----+-----+-----
  4 | A.I   | Machine Learning discussions
(1 row)

```

UPDATE 1

Exploring DELETE ... RETURNING

As we've seen, the update statement, like the INSERT statement, has the possibility to add the RETURNING keyword; this feature is also available for the delete statement:

```

forumbd=> delete from t_categories where pk=4 returning
pk,title,description;
 pk |      title      |      description
-----+-----+-----
  4 | Machine Learning | Machine Learning discussions
(1 row)
DELETE 1

```

In the next section, we'll talk about CTEs, an advanced method to return and modify data.

Exploring CTEs

In this section, we are going to talk about CTEs. This section will be split into three parts. Firstly, we will talk about the concept of CTEs; secondly, we will discuss how CTEs are implemented starting from PostgreSQL 12; and finally, we will explore some examples of how to use CTEs.

CTE concept

A CTE, or a common table expression, is a temporary result taken from a SQL statement. This statement can contain SELECT, INSERT, UPDATE, or DELETE instructions. The lifetime of a CTE is equal to the lifetime of the query. Here is an example of a CTE definition:

```

WITH cte_name (column_list) AS (
    CTE_query_definition
)
statement;

```

If, for example, we wanted to create a temporary dataset with all the posts written by the author `enrico_pirozzi`, we would have to write this:

```

forumbd=> with posts_author_1 as
(select p.* from posts p
 inner join users u on p.author=u.pk
 where username='enrico_pirozzi')
select pk,title from posts_author_1;

```

```

pk |          title
----+-----
 3 | A view of  Data types in C++
(1 row)

```

We could also write the same thing using an inline view:

```

forumdb=> select pk,title from
(select p.* from posts p inner join users u on p.author=u.pk where
u.username='enrico_pirozzi') posts_author_1;
pk |          title
----+-----
 3 | A view of  Data types in C++
(1 row)

```

As we can see, the result is the same. The difference is that in the first example, the CTE creates a temporary resultset, whereas the second query, the inline view, does not.

CTE in PostgreSQL since version 12

Starting from PostgreSQL version 12, things have changed, and two new options have been introduced for the execution of a CTE, namely `MATERIALIZED` and `NOT MATERIALIZED`. If we want to perform a CTE that materializes a temporary resultset, we have to add the `materialized` keyword:

```

forumdb=> with posts_author_1 as materialized
(select p.* from posts p
inner join users u on p.author=u.pk
where username='enrico_pirozzi')
select pk,title from posts_author_1;
pk |          title
----+-----
 3 | A view of  Data types in C++
(1 row)

```

The query written here materializes a temporary resultset, as happened automatically in previous versions of PostgreSQL. If we write the query with the `NOT MATERIALIZED` option, PostgreSQL will not materialize any temporary resultset:

```

forumdb=> with posts_author_1 as not materialized
(select p.* from posts p
inner join users u on p.author=u.pk

```

```

where username='enrico_pirozzi')
select pk,title from posts_author_1;
pk |          title
----+-----
  3 | A view of  Data types in C++
(1 row)

```

If we don't specify any option, the default is `NOT MATERIALIZED`, and this could be a problem if we are migrating a database from a minor version to PostgreSQL 12. This is because the behavior of the query planner could change, and the performance could change too.



From version 12, we have to insert the **MATERIALIZED** option if we want to have our queries display the same performance behavior that we had with the previous versions.

CTE – use cases

Let's now present some examples of the use of CTEs:

1. Firstly, we will create two new tables:
 - `t_posts`, with all the records present in the post table
 - `delete_posts`, with the same data structure as the posts table

```

forumdb=> create temp table t_posts as select * from posts;
SELECT 3

forumdb=> create table delete_posts as select * from posts limit 0;
SELECT 0

```

The starting values for the `t_posts` and `delete_posts` tables are as follows:

```

forumdb=> select pk,title,category from t_posts ;
pk |          title          | category
----+-----+-----
  1 | Indexing PostgreSQL     |      1
  2 | Indexing Mysql          |      1
  3 | A view of  Data types in C++ |      3
(3 rows)

```